WNDRVR
SUPPORT NETWORK

VXWORKS 7 NTP AND PTP PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 NTP and PTP Programmer's Guide, SR0650

25 August 2020

1. NTP OVERVIEW

1. About NTP Technology

The VxWorks® 7 NTP and PTP Programmer's Guide describes how VxWorks Network Time Protocol (NTP) conveys timekeeping information from primary servers to secondary time servers and clients over private networks and the public Internet. NTP is widely used to synchronize computer clocks on the Internet.

NTP version 4 (NTPv4) is backwards compatible with NTP version 3 (NTPv3) described in *RFC1305, Network Time Protocol (Version 3) Specification, Implementation and Analysis*, as well as previous versions of the protocol.

NTPv4 introduces new functionality to NTPv3, as described in RFC 1305, and functionality expanded from Simple NTP version 4 (SNTPv4) as described in *RFC4330, Simple Network Time Protocol (SNTP) Version 4 for IPv4, IPv6 and OSI*.

For information on NTP, refer to the following Web site:

http://www.ntp.org

Source Code Files

The source code files for NTP are in the following directory:

installDir **/vxworks-7/pkgs_v2/net/ipnet/app/ntp-***x.x.x.x***/**

Next topic: Supported RFCs for NTP on page 1

2. Supported RFCs for NTP

Wind River Network Time Protocol (NTP) Requests for Comments (RFCs) contain standards and design information defined by the Internet Engineering Task Force (IETF).

Wind River NTP complies with the following IETF RFCs:

- 5905: *Network Time Protocol Version 4: Protocol and Algorithms Specification*(obsoletes RFC1305 and RFC4330)
- RFC 5906: *Network Time Protocol Version 4: Autokey Specification*

Previous topic: About NTP Technology on page 1
Next topic: NTP Features and Functions on page 1

3. NTP Features and Functions

The Network Time Protocol (NTP) includes NTPv4 additions, operational aspects, unsupported modules, NTP daemon, commands, and the NTP and Simple Network Time Protocol (SNTP) compatibility matrix.

NTP servers and clients are configured so that time values flow towards clients from the primary servers via branching secondary servers.

NTPv4 includes a modified protocol header to accommodate the Internet Protocol version 6 address family. NTPv4 includes fundamental improvements in the mitigation and discipline algorithms that extend the potential accuracy to the tens of microseconds with modern workstations and fast LANs. It includes a dynamic server discovery scheme, so that in many cases, specific server configuration is not required. It corrects certain errors in the NTPv3 design and implementation and includes an optional extension mechanism.

NTPv4 provides the following additions to the current network time protocols:

- use of floating-point arithmetic instead of fixed-point arithmetic
- a timestamp resolution of less than one nanosecond when the double floating data type is used
- a frequency resolution of less than one nanosecond per second when the double floating data type is used
- redesigned clock discipline algorithm that improves accuracy, handling of network jitter, and polling intervals
- support for the nanokernel kernel implementation that provides nanosecond precision as well as improved algorithms
- public-key cryptography, autokey, which avoids having common secret keys
- automatic server discovery (manycast mode)
- fast synchronization at startup and after network failures (burst mode)
- new and revised drivers for reference clocks
- support for new platforms and operating systems

Operational Aspects

If a Precision Time Protocol (PTP) is present, the NTP client is not allowed to change the clock. The NTP server is able to serve this time to the NTP clients in the network.

You can set NTP parameters at configuration time from Workbench 4 or **wrtool**, at boot-time with a startup script, or at runtime with sysvars.

Specifying the Time Zone

The time zone is defined by an environment variable in the operating system. For further information on specifying the time zone, see:

- *VxWorks 7 Application Programmer's Guide:* Kernel Task Environment Variables
- *VxWorks 7 Application Programmer's Guide:* RTP Environment Variables

For information on the **TIMEZONE** environment variable format, see the ansiTime library reference at https://knowledge.windriver.com/en-us/000_Products/000/020/010/020/020/000_VxWorks_Standard_C_Libraries/090.

Unsupported Modules

The following modules from the NTP standard distribution are not supported:

- ntptrace: Trace a chain of NTP servers.
- tickadj: Set time-related kernel variables.
- ntptime: Read kernel time variables.
- ntpdsim: NTP simulator.

  The ntpdate module is replaced by the NTP daemon (ntpd).

NTP Daemon

The NTP daemon can operate in any of several modes, including client and server, symmetric and broadcast modes, and with both symmetric-key and public key-cryptography.

The NTP daemon uses a single configuration file to run the daemon in server or client mode. Important NTP files include the configuration file and the drift file. The drift file allows the daemon to correct for hardware clock skew in the absence of a connection to a more accurate upstream time server.

The NTP daemon reads the **ntp.conf** configuration file at startup to determine the synchronization sources and operating modes. You can also specify a working configuration from the command line. This may be useful when the local host is to be configured as a broadcast/multicast client, with all peers being determined by listening to broadcasts at run time.

Commands

Four commands specify the time server name or address to be used and the mode in which to operate. The address can be either a DNS name or an IP address.

server

For type **s** (remote server or peer) and **r** (reference clock) addresses, this command mobilizes a persistent client mode association with the specified remote server or local radio clock. In this mode the local clock can be synchronized to the remote server, but the remote server can never be synchronized to the local clock. Do not use this command for type **b** (broadcast) or m (multicast) addresses.

peer

For type **s** addresses, this command mobilizes a persistent symmetric active mode association with the specified remote peer. In this mode the local clock can be synchronized to the remote peer or the remote peer can be synchronized to the local clock. This is useful in a network of servers where, depending on various failure scenarios, either the local or remote peer may be the better source of time. Do not use this command for type **b**, **m** or **r** addresses.

broadcast

For type **n** and **m** addresses, this command mobilizes a persistent broadcast mode association. Multiple commands can be used to specify multiple local broadcast interfaces (subnets) and/or multiple multicast groups. Local broadcast messages are only sent to the interface associated with the subnet specified, but multicast messages are sent to all interfaces. In broadcast mode the local server sends periodic broadcast messages to a client population at the address specified, which is usually the broadcast address on the local network or a multicast address assigned to NTP. The IANA has assigned the multicast group address IPv4 224.0.1.1 and IPv6 ff05::101 (site local) exclusively to NTP, but other nonconflicting addresses can be used to contain the messages within administrative boundaries. Ordinarily, this specification applies only to the local server operating as a sender for operation as a broadcast client.

manycastclient

For type **m** addresses, this command mobilizes a **manycast** client mode association for the multicast address specified. In this case a specific address must be supplied which matches the address used on the manycastserver command for the designated **manycast** servers. The NTP multicast address 224.0.1.1 assigned by the IANA should not be used, unless specific means are taken to avoid flooding large areas of the Internet with these messages and causing a large number of replies to the sender. The manycastserver command specifies that the local server is to operate in client mode with the remote servers that are discovered as the result of broadcast or multicast messages. The client broadcasts a request message to the group address associated with the specified address and specifically enabled servers respond to these messages. The client selects the servers providing the best time. The remaining servers are discarded.

SNTP

SNTP is a subset of NTP. For information on the Simple Network Time Protocol, see the *VxWorks 7 SNTP Programmer's Guide*.

Table 1. NTP and SNTP Compatibility Matrix

	SNTP Clients V1, V2, V3, and V4	**SNTP Clients V1, V2, V3, and V4**	**NTP Client V4**	**NTP Server V4**
SNTP Clients V1, V2, V3, and V4	No	Yes	No	Yes
SNTP Servers V1, V2, V3, and V4	Yes	Yes	Yes	Yes
NTP Client V4	No	Yes	No	Yes
NTP Server V4	Yes	Yes	Yes	Yes

Previous topic: Supported RFCs for NTP on page 1

2. NTP CONFIGURATION AND BUILD

1. NTP Configuration and Build Workflow

The VxWorks 7 Netwok Time Protocol (NTP) installation includes source code but does not include pre-built kernel libraries.

Before beginning any VxWorks development, you must build the VxWorks kernel libraries (VxWorks Source Build (VSB) project). You must also build the VxWorks run-time kernel image based on the kernel libraries (VxWorks Image Project (VIP)).

For specific instructions on how to configure and build a VSB and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the VxWorks 7 Configuration and Build Guide.

For a list of available source configuration options for NTP, see NTP Source Build Configuration Options (VSB) on page 5.

For a list of available kernel configuration components and parameters for NTP, see NTP Kernel Configuration Components (VIP) on page 6.

Next topic: NTP Source Build Configuration Options (VSB) on page 5

2. NTP Source Build Configuration Options (VSB)

Network Time Protocol (NTP) configuration includes a number of source build configuration options.

You can set an optional parameter in the VxWorks source build (VSB).

Table 1. NTP VSB Options

Option	Description
Enable NTP (IPNET_NTP)	Enable NTP in the VSB.

Workbench 4

For Workbench 4, the NTP VSB options are under **net > Enable IP Networking Stack**.

To locate NTP options in the **Source Build Configuration** editor, use the find utility and enter **NTP** as the search string.

wrtool

For **wrtool**, first start the utility's shell using the -data option to load your workspace directory. For example

```
-> wrtool -data c:\WindRiver\workspace
```

Then cd to your project directory and, list the NTP VSB options as follows:

- To list the available NTP options in the VSB, use the prj vsb listAll command. For example:

```
projectDir> prj vsb listAll | grep NTP
IPNET_1_0_0_0_NTP_1_0_0_0
```

- To list the enabled NTP options in your project, use the prj vsb list command. For example:

```
projectDir> prj vsb list | grep NTP
IPNET_1_0_0_0_NTP_1_0_0_0
```

To add NTP VSB options to your project, use the prj vsb add command. For example:

```
projectDir> prj vsb add IPNET_1_0_0_0_NTP_1_0_0_0
```

To remove NTP VSB options from your project, use the prj vsb remove command. For example:

```
projectDir> prj vsb remove IPNET_1_0_0_0_NTP_1_0_0_0
```

Previous topic: NTP Configuration and Build Workflow on page 5
Next topic: NTP Kernel Configuration Components (VIP) on page 6

3. NTP Kernel Configuration Components (VIP)

Network Time Protocol (NTP) configuration includes kernel configuration components.

Use the Network Time Protocol (NTP) kernel configuration components to modify the NTP client in a VxWorks Image Project.

Table 1. NTP Components

Component	Description
Network Time Protocol (INCLUDE_IPNTP)	Include NTP in your VIP. This component also includes the INCLUDE_IPSNTPC_API and INCLUDE_IPSNTP_COMMON SNTP components. For information on the SNTP API and SNTP common configuration parameters, see the VxWorks 7 SNTP Programmer's Guide. For a VIP build, the NTP, SNTP client, and SNTP server components are mutually exclusive.
SNTP Client (API) (INCLUDE_IPSNTPC_API	This component is automatically included when you include the INCLUDE_IPNTP component. For information on the SNTP API, see the VxWorks 7 SNTP Programmer's Guide.
SNTP common configurations (INCLUDE_IPSNTP_COMMON)	This component is automatically included when you include the INCLUDE_IPNTP component. The component includes SNTP parameters common to both the SNTP client and server daemons. For information on the SNTP common configuration parameters, see the VxWorks 7 SNTP Programmer's Guide.

Workbench 4

For Workbench 4, the NTP VIP components are under **Network Components > Layer 5 (Applications) Components > NTP Components**.

To locate NTP components and parameters in the **Kernel Configuration** editor, use the **Find** utility and enter NTP as the search string.

wrtool

For **wrtool**, first start the utility's shell using the -data option to load your workspace directory. For example

```
-> wrtool -data c:\WindRiver\workspace
```

Then cd to your project directory and list the NTP VIP components as follows:

- To list the available NTP components for the VIP, use the following command:

  ```
  projectDir> prj component list all NTP
  ```

- To list the included NTP components in your VIP, use the following command:

  ```
  projectDir> prj component list NTP
  ```

  To add NTP components to your VIP, use the following command syntax:

  ```
  projectDir> prj component add componentName
  ```

  To remove NTP components from your VIP, use the following command syntax:

  ```
  projectDir> prj component remove componentName
  ```

Previous topic: NTP Source Build Configuration Options (VSB) on page 5
Next topic: Optional NTP Kernel Configuration Components on page 7

4. Optional NTP Kernel Configuration Components

Network Time Protocol (NTP) configuration includes a number of optional kernel configuration components.

A list of optional Network Time Protocol (NTP) shell command components follows.

Table 1. Optional Components

Component	Description
IPCOM NTP commands (INCLUDE_IPNTP_CMD)	Include the NTP client shell command (ntp).
IPCOM ipd commands (INCLUDE_IPD_CMD)	Include the daemon control shell command (ipd).
IPCOM syslog commands (INCLUDE_IPCOM_SYSLOGD_CM)	Include the system logging shell command (syslog).

Component	Description
IPCOM sysvar commands (INCLUDE_IPCOM_SYSVAR_CMD)	Include the system variables shell command (n).

Previous topic: NTP Kernel Configuration Components (VIP) on page 6
Next topic: NTP Configuration Parameters on page 8

5. NTP Configuration Parameters

Network Time Protocol (NTP) configuration includes a number of configuration parameters. To start the NTP daemon at boot time, you must configure one of the NTP configuration parameters.

Table 1. NTP Configuration Parameters

Parameter Name, sysvar Name, and Description	Default and Type
NTP startup parameters NTP_INIT_PARAM ipntp.init.param Specify the location of the NTP configuration file and the daemon startup options. For example: `-c /romfs/ntp.conf -g -A` -A disables authentication mode -c specifies the path and filename of the configuration file -g disables the sanity limit, (default - 1000s)	Null string
NTP config parameters NTP_CONFIG_PARAM ipntp.config.param Specify the list of parameters that are normally configured in the ntp.conf configuration file. This macro is not applicable if the NTP daemon is launched with the -c option that specifies the path and filename of the configuration file. Every parameter must be separated by a semicolon. For example: `driftfile /ram/ntp.drift; server 1.debian.pool.ntp.org iburst; server 2.debian.pool.ntp.org iburst; disable auth`	Null string

Parameter Name, sysvar Name, and Description	Default and Type
NTP config filename NTP_CONFIG_FILENAME ipntp.config.filename Specify the NTP configuration filename. This macro is not applicable if the NTP daemon is launched with the -c option specifying the path and filename of the configuration file. If NTP config filename is set to NULL, the NTP server uses the NTP config parameters values at boot time.	Null string

SNTP Common Configuration Parameters

The NTP component includes the INCLUDE_IPSNTP_COMMON component. For information on the SNTP common configuration parameters, see the VxWorks 7 SNTP Programmer's Guide.

Previous topic: Optional NTP Kernel Configuration Components on page 7
Next topic: Setting NTP Parameters at Configuration Time on page 9

6. Setting NTP Parameters at Configuration Time

You can set the Network Time Protocol (NTP) parameters from Workbench 4 or from the command line at configuration time with **wrtool**.

About This Task

Choose one of the following methods to compete this task.

Workbench 4

To locate NTP parameters in Workbench 4 with the **Kernel Configuration** editor, use the **Find** utility and enter NTP as the search string.

After the parameters display in the **Kernel Configuration** editor, you can change the parameter values.

wrtool

For **wrtool**, first start the utility's shell using the -data option to load your workspace directory. For example

```
→> wrtool -data c:\WindRiver\workspace
```

Then cd to your project directory and, list all NTP parameters within a VIP using the following command:

```
projectDir> prj parameter list NTP
```

To change NTP parameters, use the following command syntax:

```
projectDir> prj parameter set parameterName value
```

Previous topic: NTP Configuration Parameters on page 8

3. NTP SHELL COMMANDS

1. About NTP Shell Commands

The Network Time Protocol (NTP) shell commands include ntp, sysvar, ipd and syslog.

The shell commands perform the following functions:

- ntp shell command to query NTP servers or update the local time.
- sysvar shell command to modify NTP configuration parameters.
- ipd shell command to start and stop the NTP daemon.
- syslog shell command to set the priority for the NTP syslog debug messages.

To run shell commands, access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd
 [vxWorks *} #
```

To return to the kernel shell, enter the following command:

```
# C
->
```

Next topic: Using the NTP Client Shell Command on page 11

2. Using the NTP Client Shell Command

The ntp shell command allows you to query the Network Time Protocol (NTP) servers about their current state by the recommended NTP control message and also query the NTP server's time and update the local time.

The ntp shell command syntax is as follows:

```
ntp [-v] [-p] [-d] [-q] [-u] [-t timeout] [server]
```

To get help for the ntp shell command, use the following command:

```
# ntp
```

Previous topic: About NTP Shell Commands on page 11
Next topic: Setting the NTP Server System Variables on page 11

3. Setting the NTP Server System Variables

To view all NTP sysvar variables, access the command-interpreter mode from the kernel shell and execute the sysvar list ipntp command.

About This Task

For example:

```
# sysvar list ipntp
```

To create and change NTP parameters with **sysvar**, use the **sysvar set -c -o** command. For example:

```
# sysvar set -c -o ipntp.config.param "server 10.2.2.2"
sysvar: ipntp.config.param=server 10.2.2.2 ok
# sysvar list ipntp
System variables:
    ipntp.config.param=server 10.2.2.2
```

To implement changes made to NTP **sysvar** variables, you must restart the NTP daemon. See 3.4 Stopping and Starting the NTP Daemon, p.12.

SNTP Common Configuration Parameters

The NTP component includes the INCLUDE_IPSNTP_COMMON component. For information on the SNTP common configuration parameters and their system variables, see the VxWorks 7 SNTP Programmer's Guide.

Previous topic: Using the NTP Client Shell Command on page 11
Next topic: Starting and Stopping the NTP Daemon on page 12

4. Starting and Stopping the NTP Daemon

Use the ipd command to stop or start the Network Time Protocol (NTP) daemon (ipntp).

About This Task

For example:

```
# ipd list
Services:
...
ipntp                 started
```

```
# ipd kill ipntp
ipd: kill ipntp ok# #
```

```
# ipd list
Services:
...
ipntp killed
```

```
# ipd start ipntp
ipd: start ipntp ok
```


WNDRVR

To view the task information for the NTP server, run the task command and locate the ipntpd task. For example:

```
# task
NAME ENTRY TID PRI STATUS PC SP ERRNO DELAY
...
ipntpd ipntp_main 106233c8 50 PEND 102f41db 10fbfd90 0 0
```

Previous topic: Setting the NTP Server System Variables on page 11
Next topic: Setting the NTP System Log Priority on page 13

5. Setting the NTP System Log Priority

Run the syslog list command to get the current priority level for all modules.

About This Task

```
# syslog list
   syslog facility priority
   ...
   ipntp Error
```

To list the available syslog priority levels, use the following command:

```
# syslog priority *
syslog: missing <prio>
         priorities: Emerg Crit Error Warning Notice Info Debug Debug2
```

Use the following command syntax to change the priority level for NTP:

```
# syslog priority ipntp priorityLevel
```

For more information on the syslog shell command, see the VxWork 7 Network Stack Programmer's Guide.

Previous topic: Starting and Stopping the NTP Daemon on page 12

WNDRVR

4. NTP API FUNCTIONS

1. NTP API Functions

The API functions available to Network Time Protocol (NTP) applications and a brief description of each function.

For more information, see the reference entries for NTP (ipntp).

The NTP public API functions are defined in **ipntpc.c**.

Table 1. NTP API Functions

Function	Description
ipntp_changetime( )	Change the current target time.
ipntp_timediff( )	Get the time difference between the server and the target.
ipntp_updatetime( )	Obtain the time from the specified server and update the local time.
ipntp_querytime( )	Obtain the time from the specified server.
ipntp_keygen( )	Generates cryptographic data files used by the NTPv4 authentication and identification schemes.

Next topic: SNTP API Functions on page 14

2. SNTP API Functions

The Network Transfer Protocol (NTP) component includes the INCLUDE_IPSNTPC_API component.

For information on the Simple Network Transfer Protocol (SNTP) application programming interface (API) functions, see the VxWorks 7 SNTP Programmer's Guide.

Previous topic: NTP API Functions on page 14

5. PTP OVERVIEW

1. VxWorks PTP Implementation

The VxWorks Precision Time Protocol (PTP) is based on the BSD open-source implementation of the IEEE 1588-2008 PTP specification.

PTP is used to synchronize clocks throughout a computer network. It is designed for systems that require higher level of accuracy than provided by NTP, and for those without GPS support (because of inaccessible signals, or because the cost of a GPS receiver at each node is prohibitive). On a local area network, it achieves clock accuracy in the sub-microsecond range, making it suitable for measurement and control systems.

The VxWorks implementation of PTP includes the following:

- Ordinary clock support, for both master and slave mode.
- Boundary clock support.
- Layer 4 transportation.
- Layer 2 transportation.
- Synchronization of the VxWorks real-time clock to the hardware PTP clock.
- Access to the hardware PTP clock time with public APIs.
- Synchronized hardware PTP timer interrupts in one-shot and periodic modes.
- Software time stamp, with an accuracy about 10 ms.
- Hardware time stamp, with an accuracy about 1µs.

  VxWorks provides support for the following PTP and TSN hardware and protocols.

Hardware	IEEE1588v2/PTP Standard	802.1AS/gPTP Standard	802.1Qbv Standard	802.1Qbu Standard
Intel i210 NIC	Yes	Yes	Yes (Time-Specific Only)	
Intel i350 NIC	Yes	Yes		
Xilinx ZCU102	Yes	Yes		
NXP LS1028A	Yes (Ethernet Endpoint)	Yes (Ethernet Endpoint)	Yes (Time-Specific and Time-Gated) (Ethernet Endpoint)	Yes (Ethernet Endpoint)
Renesas R-Car H3	Yes			

Note: PTP for IEEE 802.1 requires a sync rate of 8 syncs per second. When the PTPD is acting as the master, use a multiple of 80 with your sysClkRateSet() call to get 8 syncs per second. For example: sysClkRateSet N * 80 (N=1,2,3,4...). Using a higher multiple for the system tick frequency can affect the precision of the VxWorks tick time (and thus the PTP soft timer), but the sync rate will remain 8 syncs per second.

Related information

TSN Clocks and Timers on page 16

2. TSN Clocks and Timers

Time-Sensitive Networking (TSN) capable devices provide synchronized time in special registers called Precision Time Protocol (PTP) hardware clock (PHC) registers. PHC registers are available for a user application through input output control (IOCTL) operations implemented in the device driver.

VxWorks provides a user friendly application programming interface (API) to access PTP time from the device. For more information see, the **tsnClkLib** library API reference entry. The library implements user access to timer interrupts based on the PHC, provided the device itself supports this capability.

Note: To enable TSN clocks and timers, the TSN and PTP VSB options must be included.

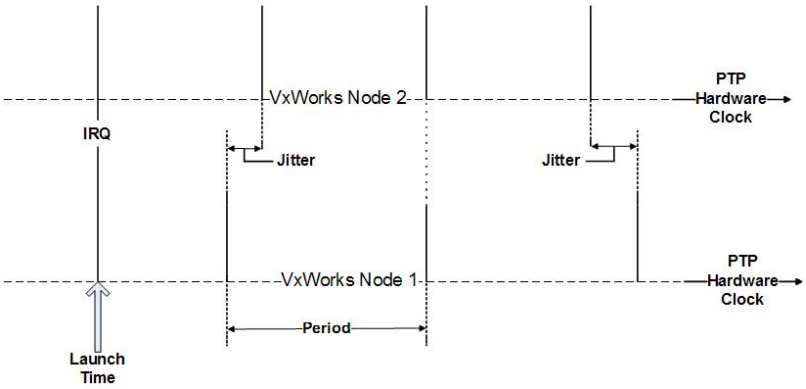
User Access to the PTP Hardware Clock

User applications acquire the identifier to the PHC with the tsnClockIdGet( ) routine and then get the PTP time with the tsnClockTimeGet( ) routine. The example below shows how to find the clock ID and then get the current PTP time into a timespec structure:

```
clockid_t cid;
struct timespec tm;
cid = tsnClockIdGet("gei",0,0);
    if (cid != 0)
    {
    ret = tsnClockTimeGet (cid, &tm);
    }
```

Synchronized Timer Interrupts

Because the PTP software keeps the PHC in sync with other devices it is possible to create timer interrupts that fire synchronously between different devices on the network. User applications may program a timer based on the PHC to fire periodically, or just once at a future time (one-shot). The main advantage of this feature is that the timer expires at the same time on all devices and the connected interrupt routine can create RTOS events that occur synchronously. The figure below shows an example of a periodic TSN timer on two different VxWorks devices:



Note that there will be some inaccuracies between the interrupts (jitter) due to hardware latencies and other sources of error. The interrupts will not drift apart over time because the clock source (PHC) stays in sync. The example below shows how to program a TSN timer to fire every millisecond (1000 Hz):

```
cid = tsnClockIdGet("gei",0,0);
    if (cid != 0)
    {
```

```
        ret = tsnTimerAllocate (cid);
        if (ret == OK)
        {
            ret = tsnClockConnect (cid, (FUNCPTR)usrTsnClockIsr, 0);
            if (ret == OK)
                ret = tsnClockRateSet(cid, 1000);
         }
     }
```

In the example above the user-defined routine, usrTsnClockIsr( ), will be called at every interrupt. Because this routine is called from an interrupt context normal VxWorks API restrictions apply. Use tsnClockEnable( ) to enable the timer. For additional information, refer to the specific API documentation.

Clock Jitter Reduction

There are several ways to optimize the jitter of the TSN clock as mentioned below (ordered by the effects):

1. To reduce jitter with a GEI driver, set the interruption throttle interval as small as possible within the capability of the system. To set this interval, update the VIP parameter **TSN_CLK_INT_THROTTLE_INTERVAL**. The default value of this parameter is -1, which indicates that this feature is disabled. Setting it to a positive value will enable this feature. The unit of the value is in µs (microsecond). As per some tests, on a mainstream Intel board the value can be set to 1. What this means is that the interval between two interruptions is not larger than 1µs or **TSN_CLK_INT_THROTTLE_INTERVAL**.
2. In a multicore environment, it is possible to allocate a dedicated CPU core for the TSN clock interruption process. The API tsnClockIntReroute( ) is designed for this purpose. Use this API to reroute the interruption of one NIC to a specified CPU core.
3. Disable the power management feature by excluding the folder **FOLDER_PWR_MGMT** from the VIP.

Related information

VxWorks PTP Implementation on page 15

6. VXWORKS PTP CONFIGURATION

To use PTP, you must first create a VSB configured with PTP options, then create a VIP based on VSB, and configure the VIP with PTP option. You can configure PTP for the IEEE1588 v2 default configuration, or for the IEEE 802.1AS specification, and for ordinary or boundary clocks.

VSB Configuration

COMPONENT_IPPTP

PTP code layer.

Note: Include **IPNET_CLOCK** to synchronize real-time clock with the PTP clock, and to generate interrupts synchronized with the PTP clock. See TSN Clocks and Timers on page 16 for more information.

Note: PTP is based on International Atomic Time (TAI, from the French name temps atomique international) and Time Zones are based on UTC (Coordinated Universal Time). Conversion between TAI and UTC is not supported and requires additional user configuration.

Note: Messaging uses multicast; unicast is not enabled, therefore slaves receive every message communicated in the multicast group.
There is no mechanism to filter messages on either the hardware or software levels.

VIP Basic Configuration

INCLUDE_1588_CLOCK

Synchronizes the real-time clock with the PTP clock.

INCLUDE_IPPTP

Provides Precision Time Protocol support. For information about binding PTP to Ethernet interfaces and so on, see VxWorks PTP APIs on page 21 and the **ipptp** library API reference entry.

PTP_CLK_TASK_PRIORITY

Sets PTP task priority.

INCLUDE_IPPTP_SHOW

Provides the runtime status for the PTP daemon. For information about the show routine and other troubleshooting functions, see VxWorks PTP APIs on page 21 and the **ptpd** library API reference entry.

SELECT_IPPTP_PROFILE

PTP profile selection. Select either the IEEE 1588 default or the IEEE 802.1AS profile.

- **IPPTP_DEFAULT_PROFILE**

  PTP IEEE 1588 default profile.

- **IPPTP_802_1AS_PROFILE**

  PTP IEEE 802.1AS profile.

WNDRVR

SELECT_IPPTP_CLOCK_TYPE

Select an ordinary or boundary clock type.

- **SELECT_IPPTP_ORDINARY_CLOCK**

  Selects the ordinary clock type. To use this feature, create an initialization function to call ptpdAdd( ) multiple times with the required Ethernet interface names, and then call ptpdStart( ). Alternatively, you can automatically start the clock on startup by including **INCLUDE_IPPTP_ORDINARY_CLOCK**. For more information, see VxWorks PTP APIs on page 21 and the **ipptp** library API reference entry.
- **SELECT_IPPTP_BOUNDARY_CLOCK**

  Select the boundary clock. Includes the boundary clock that requires 2 or more PTP interfaces. To use this feature, call ptpdAdd( ) to add one slave clock and one or more master clocks. Then call ptpdStart( ) to start the clocks.

For more information, see INCLUDE_1588_CLOCK and other components in the VxWorks 7 Time-Sensitive Networking Programmer's Guide.

Note: Customer profile and power profile are not supported.

VIP Configuration for Ordinary Clock Support

INCLUDE_IPPTP_ORDINARY_CLOCK

Automatically starts the clock on start up.

INCLUDE_IPPTP_ORDINARY_CLOCK configuration options:

- **PTP_ORDINARY_IF_NAME** sets the PTP interface name. The default is "gei0".
- **PTP_ORDINARY_TRANSPORT_LAYER** sets the PTP transport layer (layer 2 or 4). The default is 2.
  Configuring Layer 2: For 802.1AS use MAC address: 01-80-C2-00-00-0E; for non-802.1AS use MAC address: 01-1B-19-00-00-00
  Configuring Layer 4: IP address: 224.0.1.129 is used by all except peer delay messages; IP address: 224.0.0.107 is used by peer delay messages. Event and General Sockets join both multicast groups.
- **PTP_ORDINARY_MASTER_MODE** sets master mode (**0**) or slave mode (**1**, the default).
- **PTP_ORDINARY_DELAY_MECHANISM** sets the ordinary clock delay mechanism to P2P mode (**TRUE**, the default) or E2E mode (**FALSE**).
- **PTP_ORDINARY_ONE_STEP_MODE** sets the ordinary clock step-mode to one-step (**TRUE**) or two-step (**FALSE**, the default).
- **PTP_ORDINARY_PRIORITY** sets the ordinary clock priority when the clock is configured as the master clock. The default is 128.

  Note: Both priority1 and priority2 are set to the value of PTP_ORDINARY_PRIORITY.
- **PTP_ORDINARY_PDELAY_REQ_INTERVAL** sets the PTP ordinary clock delay request interval, as a reciprocal of *n* times the power of 2 (from 1 to -4, which means from 0.5 PPS to 16 PPS). The default is 0.
- **PTP_ORDINARY_DEBUG_LEVEL** sets the default debug level (0-6). The default is 0.

VIP Configuration for Boundary Clocks

The VxWorks PTP implementation provides for configuration with boundary clocks. Each of the boundary clock components is similar to **INCLUDE_IPPTP_BOUNDARY_CLOCK_X**, where *X* is the boundary clock number. Clock 1 is the slave clock and clocks 2-4 are master clocks.

WNDRVR

SELECT_IPPTP_BOUNDARY_CLOCK

Select the boundary clock. Includes the boundary clock that requires 2 or more PTP interfaces. To use this feature, call ptpdAdd( ) to add one slave clock and one or more master clocks. Then call ptpdStart( ) to start the clocks.

INCLUDE_IPPTP_BOUNDARY_CLOCK_1

Selects and automatically starts the slave boundary clock 1.

INCLUDE_IPPTP_BOUNDARY_CLOCK_1 clock configuration options:

- **PTP_BOUNDARY_CLK1_IF_NAME** sets the interface name. The default is *gei0*.
- **PTP_BOUNDARY_CLK1_TRANSPORT_LAYER** sets the PTP transport layer to 2 or 4 (the default is 2).
- **PTP_BOUNDARY_CLK1_DELAY_MECHANISM** sets the delay mechanism to P2P mode (**TRUE**, the default) or E2E (**FALSE**).
- **PTP_BOUNDARY_CLK1_ONE_STEP_MODE** sets the step mode to one-step (**TRUE**) or two-step (**FALSE**, the default).
- **PTP_BOUNDARY_CLK1_PRIORITY** sets the boundary clock priority, when configured as the master clock. The default is 128.
- **PTP_BOUNDARY_CLK1_PDELAY_REQ_INTERVAL** sets the PTP boundary clock pdelay request interval, as the reciprocal of *n* times of power of 2 (from 1 to -4, which means from 0.5PPS to 16PPS). The default is 0.
- **PTP_BOUNDARY_CLK1_DEBUG_LEVEL** sets the default debug level, from 0 to 6. The default is 0.

7. VXWORKS PTP APIS

PTP provides APIs for binding, starting, and monitoring the PTP daemon. Other network APIs are used for controlling the MUX or device, and for setting socket options.

ipptp Library

The **ipptp** library can be used in two different ways to start PTP daemon tasks for Ethernet interfaces. The first involves using ptpdAdd( ) calls to bind PTP to each of the Ethernet interfaces, and then calling ptpdStart( ) to start all of the PTP daemon tasks. The second method is to call ptpdAdd( ) for a given interface, to call ptpdStartClock( ) to start a PTP daemon task for that interface, and then to repeat this process for each of the other interfaces.

API	Description
ptpdAdd( )	Binds PTP to Ethernet interfaces. For a boundary clock, it should be called once for each task that is running on an Ethernet card. Note: If you set the slaveMaster parameter as 2 (BMCA), and this node is not the best master, it will enter slave or passive mode, depending on its configuration, and sync with the best available master.
ptpdStart( )	Starts PTPD instances on all interfaces.
ptpdSetDebugLevel( )	Sets the PTP tasks debug level. Used in tracking the log output for Ethernet cards.
ptpdStartClock( )	Starts the specified PTPD interface.
ptpdStop( )	Stops the specified PTPD interface.
ptpdClearAll( )	Clears all the attached PTPD interfaces.

ptpd Library

API	Description
pptpdBooleanParamSet( )	Sets a boolean parameter value for PTPD configuration variables *stepForce* , *stepOnce* and *noResetClock* .
ptpdIntegerParamSet( )	Sets an integer parameter value for PTPD configuration variable *resetJitter* .
ptpdDoubleParamSet( )	Sets a double parameter value for PTPD configuration variables *servoKP* and *servoKI* .
ptpdMasterClockIdGet( )	Gets the clock ID of the grand master or parent master.

API	Description
ptpdStatusShow( )	Shows the PTPD runtime status.
ptpdGetSyncStatus( )	Gets the PTPD synchronization status. The supported PTPD status types are PTP_INITIALIZING=1, PTP_FAULTY, PTP_DISABLED, PTP_LISTENING, PTP_PRE_MASTER, PTP_MASTER, PTP_PASSIVE, PTP_UNCALIBRATED, and PTP_SLAVE.

Additional API Information

For additional information, see the **ipptp** and **ptpd** library API reference entries.

8. GPTP

1. Overview

VxWorks provides support for the open source implementation of Generalized Precision Time Protocol (gPTP), so that the devices can remain synchronized with high precision (nanoseconds).

For more information, see https://github.com/Avnu/gptp.

gPTP has been tested on IA and ARM(ZCU102 and LS1028A) platforms.

Limitations

- Only one gPTP daemon is allowed to run at a time.
- Once the daemon is started, it cannot be manually stopped.
- The log output cannot be controlled at run-time. You can customize the log output (turn on/off) by commenting or uncommenting the following macros defined in the file *installDir* **/vxworks-7/pkgs_v2/net/gptp/gptp/common/gptp_log.hpp** :

```
#define GPTP_LOG_CRITICAL_ON 1
#define GPTP_LOG_ERROR_ON 1
#define GPTP_LOG_EXCEPTION_ON 1
#define GPTP_LOG_WARNING_ON 1
#define GPTP_LOG_INFO_ON 1
#define GPTP_LOG_STATUS_ON 1
#define GPTP_LOG_DEBUG_ON 1
#define GPTP_LOG_VERBOSE_ON 1
```

- gPTP only supports with 1G or 100M bps and Duplex.

2. gPTP Configuration and Build

You must create a VxWorks image project (VIP) and associate it with a VSB project that includes the Generalized Precision Time Protocol (gPTP) layer.

Before You Begin

To create, configure and build a VSB from the command line, you must set up the wrtool utility appropriately by pointing it to the right workspace. For more information, see the *VxWorks 7 Configuration and Build Guide: Creating a VxWorks Source Build Project from the Command Line*.

Procedure

1. Create the VSB project.

```
$ vxprj vsb create -force -S -bsp itl_generic -cpu CORE -lp64 vsb_gptp
$ cd vsb_gptp
```

2. Enable gPTP.

```
$ vxprj vsb config -s -add _WRS_CONFIG_GPTP=y
```

3. Build the VSB project.

```
$ make -j [thread number]
```

4. Create a VIP based on the VSB you created in step 1.

```
$ vxprj create -force itl_generic llvm -vsb vsb_gptp vip_gptp
$ cd vip_gptp
```

5. You can start the gPTP daemon automatically when VxWorks starts or you can start the daemon manually on the shell. Follow the steps below to start the gPTP dameon manually:
   1. Add the component **INCLUDE_GPTP** to enable gPTP.

   ```
   $ vxprj component add INCLUDE_GPTP
   ```

   ```
   $ vxprj parameter set GPTP_TASK_PRIORITY 80 (optional)
   ```

   2. (Optional) Add the component **INCLUDE_SHELL** to include the kernel shell.

   ```
   $ vxprj component add INCLUDE_SHELL
   ```

   3. Build the project.

   ```
   $ vxprj build
   ```

6. To start the gPTP daemon automatically, follow the steps below:
   1. Add the components to enable gPTP and ipnet configuration.

   ```
   $ vxprj component add INCLUDE_GPTP_START
   $ vxprj parameter set GPTP_PRIORITY_ONE 248
   $ vxprj parameter set GPTP_DEVICE_NAME '"gei1"'
   $ vxprj component add INCLUDE_IPNET_IFCONFIG_2
   $ vxprj parameter set IFCONFIG_2 '"ifname gei1", "devname gei1"'
   ```

   2. Build the project.

   ```
   $ vxprj build
   ```

3. Running the gPTP Daemon

gPTP dameon can be run manually on the kernel shell.

Procedure

1. Run the command daemon_cl on the kernel shell with appropriate parameters, to start the gPTP daemon. Some examples of running the command below:

Normal configuration in BMC mode. 1 can be replaced with the priority value expected.

```
daemon_cl "gei1 -R 1"
```

Start grandmaster for automotive profile.

```
daemon_cl "gei1 -V -GM"
```

Start grandmaster for automotive profile with test mode.

```
daemon_cl "gei1 -V -E -GM"
```

Initial synchronization interval(Log base 2. -3 = 125ms) for automotive profile mode.

```
daemon_cl "gei1 -V -E -INITSYNC -3"
```

To find out the usage of the command daemon_cl, run it on the shell without providing any parameters.

Following configurations are supported for gPTP:

- VxWorks Master ↔Linux Slave
- Linux Master ↔VxWorks Slave

2. To communicate with a Linux daemon:
   1. Build the Linux files.

   ```
   git clone https://github.com/AVnu/gptp.git
   ```

   ```
   cd gptp/linux/build/
   ```

   ```
   make clean all
   ```

   You can find daemon_cl under the folder **gptp/linux/build/obj**

   2. Run the grand master. Make sure to replace **ens5** with a valid network interface name.

   ```
   sudo ./daemon_cl ens5 -R 1
   ```

   3. Run the slave. Make sure to replace **enp3s0** with a valid network interface name.

   ```
   sudo ./daemon_cl enp3s0 -R 248
   ```

   Note: To tune the convergence time of gPTP, use the parameter INITSYNC of the command daemon_cl. Based on the tests performed by Windriver with two IA (CORE) boards with i210 NICs, when INITSYNC is -3(sync 8 times per second), the offset between the master and the slave can reach <100ns in 2 seconds.